

Offline Evidence, Online Admission: Action-Bound Human Authorization Under a Signature-Soliciting Adversary

Iman Schrock
EMILIA Protocol, Inc.
`team@emiliaprotocol.ai`

Preprint. August 2026.

Abstract

An agent can hold a valid session and broad delegated authority while still needing a human decision for one consequential action discovered during execution. A common response is to create a signed approval artifact that can be checked later without contacting the approval service. Two different properties are often attributed to that artifact: that it is bound to the action the human approved, and that it can be used only once. This paper separates them.

We define an action-bound authorization receipt for an adversary that controls the network and the approval orchestrator and may ask an honest approver to sign arbitrary actions. Under standard signature and hash assumptions, offline verification gives action agreement: a receipt accepted for action a contains an uncompromised approver’s signature over a canonical commitment to a . It does not give global one-time admission. We show that a static artifact presented to two isolated, otherwise correct verifiers can be accepted once by each. Preventing that replay requires a shared atomic consumption domain, equivalent coordination, or a construction that restricts the artifact to one admission domain.

We encode both boundaries in Tamarin. The core model verifies user-verification-gated action agreement and local injective acceptance, and finds the expected replay when consumption is omitted. A second model verifies injectivity inside each consumption domain while falsifying global injectivity across domains, even without key compromise. The result is a narrow placement rule: signatures make authorization evidence portable; state at the execution boundary makes admission one-time.

1 Introduction

An AI agent may begin a task under a valid session and discover, only after it has read data or planned a sequence of calls, that one step requires a human decision. Examples include changing a payment destination, deleting a production resource, or releasing confidential data. The question at that point is not simply whether the agent has a credential. It is whether a particular human authorized a particular action, and whether that authorization may still admit execution.

Digital signatures are a natural part of the answer. If the human signs a canonical commitment to the action, a later verifier can test the signature and the action binding without trusting the service that routed the approval. This is useful when the operator of the approval service is also the party that would otherwise produce the audit record. A portable signed artifact turns one claim from an operator-controlled database entry into independently checkable evidence.

The same artifact is sometimes also described as “single use” or “exactly once.” That language crosses a boundary. Signature verification is a pure computation over an artifact, a public key, and an expected message. Pure verification has no memory. If two isolated verifiers receive the same valid artifact, each sees an unused artifact in its own local state. Both can accept. No signature algorithm can tell one verifier what another verifier already did.

This paper studies that separation under a deliberately strong adversary. The adversary controls the network

and the orchestration service. It can choose the actions shown to the signing interface and collect honest signatures on as many other actions as it can persuade the human to approve. This is a signature-soliciting adversary, not merely an attacker trying to forge a signature without access to the signer. The construction should therefore prevent a real signature for action a' from being transplanted onto action a .

The contributions are:

1. A small action-bound receipt construction and acceptance procedure that separate portable evidence from stateful admission.
2. A security model in which an untrusted orchestrator can solicit honest signatures over attacker-chosen actions.
3. An action-agreement argument under canonical encoding, collision resistance, signature unforgeability, and uncompromised key pinning.
4. A simple impossibility result for global one-time admission by a static artifact at isolated verifiers.
5. Two executable Tamarin models that verify the positive action-binding and local-consumption properties and expose the two replay boundaries.

The paper does not propose that a signed human interaction is itself an authorization decision. A policy or authorization service decides whether the evidence is sufficient. An executor then admits or refuses the exact action. The receipt is an input to those decisions and later evidence of what was signed.

2 System and Adversary Model

2.1 Parties

The system has four roles.

Approver H A human associated with a signing key pair (sk_H, pk_H) . The signing device requires a local user-verification event before producing the signature.

Orchestrator O
The service that prepares an action, requests approval, and transports artifacts. It is not trusted for evidence integrity.

Authorization service A
The policy decision point. It may accept a valid receipt as one input to a grant. Its internal policy is outside the cryptographic construction.

Executor X The component able to cause the consequential effect. It checks the exact action and atomically records consumption before commitment.

The roles may be operated by the same organization, but the trust boundaries do not disappear when software is colocated. In particular, a database row written by O is not treated as independent evidence of what H signed.

2.2 Signature-soliciting adversary

The adversary controls all network traffic and the orchestrator. It may delay, drop, reorder, duplicate, and replay messages. It chooses the action in any signing request and receives every resulting receipt. It can therefore obtain valid signatures over arbitrary actions that the human actually approves.

The adversary does not initially know sk_H . Key compromise is represented by an explicit reveal event. Security statements are conditioned on no reveal before the relevant acceptance. The relying party obtains the association between the approver identifier and pk_H through a trusted pinning process. The directory or identity proof that establishes that association is not analyzed here.

This model is stronger than one that assumes the signer participates only in an honest protocol run. The construction must remain action-bound even when the attacker possesses honest signatures for many nearby actions.

2.3 Action model

An action a is a typed object containing every field that can change the security-relevant effect. For a payment this would include the asset, amount, destination, source account, and relevant execution constraints. The action has a deterministic canonical encoding $\text{Enc}(a)$. JSON deployments can instantiate this requirement with JCS [6], provided the action schema also fixes number, string, and omitted-field semantics.

Definition 1 (Exact action). *Two actions are equal for authorization if and only if their canonical encodings are byte-for-byte equal. An omitted material field is therefore a defect in the action schema, not a fact repaired by the signature.*

The construction assumes that Enc is deterministic and injective over valid actions. A parser differential or an ambiguous schema violates this assumption. It must be addressed by canonicalization rules and conformance tests, not by the symbolic signature proof.

3 Construction

3.1 Signed authorization context

For domain-separation tag t , action a , fresh nonce n , audience aud , policy reference p , and validity interval w , define

$$c = \text{Enc}(t, \text{H}(\text{Enc}(a)), n, aud, p, w). \quad (1)$$

The signing device renders the action from the same canonical object used to compute the hash. After local user verification, it returns

$$\sigma = \text{Sign}_{sk_H}(c). \quad (2)$$

The authorization receipt is

$$R = (a, n, aud, p, w, H, \sigma, keyref). \quad (3)$$

The full action travels with the receipt. This avoids asking a later verifier to recover security-relevant parameters from operator prose. The key reference lets the verifier resolve a pinned public key and its applicable status information.

3.2 Offline evidence verification

Given R , a relying party reconstructs c , resolves the pinned key, and checks:

1. the action is well formed and canonically encoded;
2. the expected audience and policy reference equal the signed values;
3. the relevant validity and key-status inputs satisfy local policy; and
4. $\text{Verify}_{pk_H}(c, \sigma) = 1$.

The signature and binding checks are offline if the verifier already has the necessary key and status material. The word “offline” does not make those inputs current. A verifier using an old revocation snapshot learns only what was valid under that snapshot.

3.3 Admission and consumption

Verification produces evidence. Admission is a separate state transition. Let D be an authoritative consumption domain and let

$$k_R = (H, n, aud) \quad (4)$$

be the receipt's consumption key. The executor performs one atomic operation:

$$\text{Accept}_D(R, a_{exec}) = \begin{cases} \text{refuse,} & a_{exec} \neq a, \\ \text{refuse,} & \text{verification or policy fails,} \\ \text{refuse,} & k_R \text{ is already consumed in } D, \\ \text{commit,} & \text{Consume}_D(k_R) \text{ succeeds atomically.} \end{cases} \quad (5)$$

The consumption record is written only for the action that is about to be committed. A refusal does not spend the authorization. The effect itself may need an idempotency key or a transaction in the system of record. A consumption record proves at-most-once admission by this gate; it does not prove an exactly-once downstream physical or financial effect.

3.4 Claim separation

Table 1 states what each mechanism contributes. The separation is important because a receipt can remain valid evidence after it has been consumed. Consumption changes whether the action may be admitted again; it does not make the historical signature false.

Mechanism	Establishes	Does not establish
Signature over c	Integrity and origin under the pinned key	Human identity, comprehension, policy sufficiency
Action hash and canonical encoding	Binding to the exact encoded action	Completeness of the action schema or honest display
Fresh nonce	Distinguishes authorization instances	Erasure or single use after disclosure
Audience and policy commitment	Scope of intended reliance	Current authority or policy correctness
Atomic consumption in D	At-most-once admission inside D	Global replay prevention outside D or downstream effect

Table 1: Mechanisms and bounded claims.

This also explains why verification and admission should have different outputs. An offline verifier can return that the signature and action binding are valid under supplied trust inputs. Only the authoritative executor can return that the authorization is currently unused in its domain and has been admitted.

Orchestrator	Approver device	Authorization service and executor
Prepare canonical a	→ Render a ; require user verification	
Receive R	← Sign c and return R	
Present R, a	→	→ Verify evidence; decide policy; atomically consume and commit

4 Security Properties

4.1 Action agreement

Definition 2 (Action agreement). *If a relying party accepts a receipt naming H for action a , and H 's signing key was not compromised before acceptance, then H previously produced a signature over a context containing $H(\text{Enc}(a))$ and the same nonce, audience, policy reference, and validity interval.*

This definition is intentionally about the signed bytes. It does not claim that the human understood the action, that the display was honest, or that policy should authorize it.

Security experiment. The challenger generates (sk_H, pk_H) and gives pk_H to the adversary. The adversary has a signing oracle that takes a valid action and context parameters, records a user-verification event, and returns the corresponding receipt. This is the cryptographic abstraction of an orchestrator that can solicit signatures on chosen actions. The adversary wins if it outputs (R, a) such that evidence verification accepts under pk_H , but the exact context reconstructed from a does not appear in the signing-oracle transcript. We call this experiment **ABind**.

The experiment is not intended to model whether the human should have approved an oracle query. The oracle records that the approver did sign it. The security goal is that a receipt from one query cannot be presented as evidence for a different action or context.

Proposition 1 (Cross-action non-transferability). *Assume that the signature scheme is existentially unforgeable under chosen message attack, H is collision resistant, Enc is injective over valid actions, the verifier reconstructs the signed context from the received action, and the pinned public key belongs to the named approver. An adversary that obtains honest signatures on actions of its choice cannot cause acceptance for a new action a unless the approver signed the corresponding context, the signing key was compromised, or one of the assumptions fails.*

Proof. Suppose a receipt is accepted for a without a prior signature on its context. The verifier accepted a signature under pk_H for the reconstructed c . If the signature was not previously returned on c , this is a signature forgery. If it was returned for a context containing a different action a' , equality of the verified contexts requires either $\text{Enc}(a) = \text{Enc}(a')$ or a hash collision. By injectivity, the first case gives $a = a'$. The remaining cases contradict the stated assumptions. $\square$

The proposition gives a direct reduction from a successful **ABind** adversary to either signature forgery, a hash collision, or an encoding collision. Domain separation in t also prevents a signature produced for another message type from being interpreted as an authorization receipt.

Fresh nonces stop one approval instance from being confused with another, but a nonce does not erase a valid signature after use. That property needs state.

4.2 Local injective admission

Definition 3 (Local injective admission). *Within a consumption domain D , no two distinct commitment events accept the same receipt consumption key k_R .*

This follows from a linearizable compare-and-set, uniqueness constraint inside the same transactional database, or equivalent atomic mechanism. Merely writing a log entry after dispatch does not provide the property because two concurrent requests may both pass the pre-check.

The ordering of checks matters. The executor first validates that the nonce is available without spending it. It then validates the action, audience, policy, and other required evidence. Only when it commits to the effect does it finalize consumption. A refusal must not consume. A deployment may use a short reservation while checks run, but every refusal and abandoned attempt must release that reservation. Otherwise a rejected action can permanently destroy a valid grant.

4.3 Why offline artifacts cannot be globally single use

Proposition 2 (No global single use at isolated verifiers). *Let R be a static, valid authorization artifact addressed to an admission set containing two verifiers V_1 and V_2 . Assume the verifiers start from states in which R is locally unused, do not communicate before deciding, and each must accept a fresh valid artifact in an otherwise valid request. No deterministic or randomized local verification procedure can guarantee that at most one verifier accepts R while preserving acceptance at both verifiers when each is run alone.*

Proof. Present R to both verifiers. The local view of V_1 is indistinguishable from an execution in which only V_1 receives the fresh artifact. Correctness therefore requires V_1 to accept. The same argument applies to V_2 . With deterministic verifiers, both accept. With randomized verifiers that accept a fresh valid artifact with nonzero probability, there is a nonzero probability that both accept. Preventing the joint event requires information that changes one local view, such as shared state, coordination, a unique online token exchange, or an artifact whose audience permits only one admission domain. $\square$

Audience restriction changes the set over which the claim is made; it does not make offline state global. If several replicas share one audience and may each cause the effect, they still need a common atomic consumption record or an idempotent system-of-record operation.

4.4 Available design choices

Proposition 2 does not say that useful systems are impossible. It identifies the resource each design must spend. Four common choices are:

1. **Shared atomic state.** All executors consult one linearizable consumption domain. This preserves flexible routing but adds an online dependency.
2. **Single-domain addressing.** The signed audience names one executor or one domain with an internal atomic store. This preserves offline evidence verification but gives up transparent admission at other domains.
3. **Idempotent effect key.** The system of record rejects a second operation carrying the same key. This moves consumption to the effect boundary and is often the strongest placement when the effect system supports it.
4. **Detect after use.** Independent domains accept offline and later reconcile duplicate use. This provides double-use evidence, not at-most-once prevention, and is inappropriate when the second effect is irreversible.

Any claim of single use should therefore name both the consumption key and the domain over which uniqueness is enforced. “The receipt is non-replayable” is incomplete without those two facts.

5 Symbolic Analysis

5.1 Method

We use Tamarin Prover 1.10.0 with its standard symbolic signing and hashing theories and Maude 3.4 [2]. The network is controlled by the Dolev-Yao adversary [1]. The public key is public. A relying party pins it out of band. The action in a signoff request comes from an adversary-controlled input, so the model permits signatures over arbitrary attacker-chosen actions.

User verification is represented by a linear fact consumed by the signing rule. This models the requirement that the authenticator gate each signature on a local verification event, as in a WebAuthn-style ceremony [7]. It does not model authenticator firmware, biometric quality, or display fidelity. Canonical encoding is represented by injective symbolic terms, and signatures have perfect symbolic cryptography.

The central signing rule emits the tuple containing the action, nonce, and signature onto the adversarial network:

```
rule Human_Sign_Receipt:
  [ UVDone(H,a,n), !HumanLtk(H,ltk) ]
  --[ Signed(H,a,n) ]->
  [ Out(<a,n,sign(<'ep_signoff_v1',h(a),n>,ltk)>) ]
```

The action is not generated by an honest role. It originates in an $\text{In}(\mathbf{a})$ premise in the request rule, which means the adversary chooses it. This detail is load-bearing: the action-agreement lemma is checked even when the adversary can obtain honest signatures on other chosen actions.

5.2 Core model

The core model has two acceptance rules. The first checks the signature but does not consume the receipt. The second emits a consumption event constrained to occur once. Five lemmas are evaluated:

Lemma	Expected	Result
Honest user-verified receipt is executable	witness	verified, 8 steps
Acceptance implies prior user verification and signature for the exact action	all traces	verified, 12 steps
Honest signatures for other actions cannot be transplanted	all traces	verified, 12 steps
Checked acceptance is injective with consumption	all traces	verified, 6 steps
Unchecked acceptance is injective	false	falsified, 10 steps

Table 2: Results for `ep_receipt_core.spthy`. The last lemma is retained as a negative control.

The falsified lemma has the expected trace: the adversary obtains one honest receipt and presents it twice to the unchecked acceptance rule. The trace does not forge a signature or change the action.

5.3 Split-domain model

The second model replaces the core model’s single global consumption restriction with a restriction indexed by domain D . Each domain is internally atomic, but there is no fact or rule that coordinates different domains.

Lemma	Expected	Result
Honest user-verified receipt is executable	witness	verified, 8 steps
Action binding survives local consumption	all traces	verified, 6 steps
Acceptance is injective inside one domain	all traces	verified, 2 steps
Acceptance is globally injective across domains	false	falsified, 8 steps

Table 3: Results for `ep_receipt_split_domains.spthy`. The final counterexample contains no key-reveal event.

The counterexample is the symbolic form of Proposition 2. One honest signature is delivered to two relying parties operating in different consumption domains. Each domain consumes it once. Local injectivity holds, while global injectivity fails. This is not a break of action binding. It is a failure to coordinate state.

5.4 Interpretation

The models establish three points under their assumptions.

1. Giving the adversary access to honest signatures over chosen actions does not defeat exact action binding.
2. A signature check without consumption permits same-action replay.
3. Consumption state that is atomic only locally does not compose into a global at-most-once property.

The models do not prove computational security of a particular signature suite. They also do not prove the directory, canonical parser, clock, authorization policy, or downstream effect system. Those exclusions are part of the result, because each excluded component supplies an assumption the theorem actually uses.

6 Placement in an Agent Authorization Flow

Action-bound approval is most useful when the need for human authorization is discovered after an agent has begun work, a gap also identified in the current AI-agent authorization Internet-Draft [11]. The agent pauses

before the consequential step. The orchestrator constructs the exact action object and requests the receipt. An authorization service validates the receipt together with delegation, identity, risk, and policy inputs. If it grants authority, the executor still compares the action at commitment time and atomically consumes the authorization.

This placement separates four questions that are easy to collapse:

1. **Evidence validity:** does the signature verify over the exact action?
2. **Policy sufficiency:** does this receipt, with other evidence, satisfy the relying party’s rule?
3. **Authorization:** does the policy decision point grant this action?
4. **Admission:** may the executor commit it now, and has this grant already been consumed in the relevant domain?

The receipt answers the first question. It can be an input to the second and third. It does not replace the fourth. A deployment that verifies the artifact only in agent-side middleware can protect against agent error and network replay through that path, but an operator who controls an alternate execution path can bypass it. Strong at-most-once admission requires that every protected path meet the authoritative consumer or system of record.

7 Related Work

Transaction authorization and what-you-see-is-what-you-sign systems bind a user approval to transaction data rather than to a general authenticated session [5]. The present construction applies the same exact-message discipline to an agent action discovered during execution, and places the operator that solicits the signature inside the adversarial model.

OAuth Rich Authorization Requests [8] and transaction tokens [10] carry structured authorization details and workload identity across services. They are useful inputs to the policy and delegation layers. They do not by themselves provide an approver-held signature over the final action or a global consumption service. OpenID Client-Initiated Backchannel Authentication [9] begins with a client request and provides an out-of-band authentication and authorization flow. The problem studied here is narrower: the evidence artifact and admission rule when the exact need for human authorization appears during an agent run.

Formal analyses of OAuth 2.0 [3] and FIDO2 [4] show the value of stating web and authenticator assumptions explicitly. This paper treats WebAuthn-style user verification and key custody as assumptions and verifies the receipt protocol around them. Tamarin has been used for symbolic analysis of authentication, authorization, and payment protocols. Our models use its trace logic to retain both the positive agreement theorem and the expected replay counterexamples.

Recent work on consent integrity for black-box agents [12] focuses on the trusted path from a boundary action to the human display and back to execution. That problem is complementary. The present paper assumes a fixed action object, then asks what the resulting signature proves to a later verifier and where single-use admission can be enforced.

Offline payment and electronic cash systems [13, 14] address double spending with online checks, tamper-resistant hardware, or later detection and identification. The same fundamental distinction appears here in a simpler setting: a portable cryptographic object can be copied. Preventing two isolated executors from acting on the copy needs state or a restricted execution locus.

8 Limitations

The construction has several explicit limits.

- **Human perception.** The signature covers canonical bytes, not human understanding. A compromised renderer can mislead the approver unless the signing surface is independently trusted or the action is displayed through a protected path.

- **Identity binding.** The symbolic proof pins a public key. It does not prove that a directory correctly maps that key to a natural person or role.
- **Key and status freshness.** Offline verification can use stale key status. Current revocation generally requires a current authenticated input.
- **Canonicalization.** The proof assumes an injective encoding. Parser differentials, omitted material fields, and unsafe numeric representations are implementation concerns that need separate testing.
- **Policy.** A valid receipt may be insufficient under policy, or may authorize a bad action. Cryptographic validity is not business correctness.
- **Effects.** Atomic consumption provides at-most-once admission in its domain. It does not prove that a remote effect occurred, occurred once, or can be reversed.
- **Availability and coercion.** The adversary can deny service, and the construction does not detect coercion or rubber-stamping.

9 Conclusion

Action-bound signatures and one-time execution solve different problems. A signature over a canonical action lets a third party verify, without trusting the approval operator, which bytes an uncompromised approver key signed. That property survives an adversary that can solicit honest signatures over other actions. It is portable and can be checked offline.

Single use is not portable in the same way. A receipt can be copied, and isolated verifiers cannot learn each other’s decisions from a static artifact. At-most-once admission therefore belongs at a shared atomic consumption boundary or an equivalent uniquely addressed execution point. The Tamarin models preserve this line: they verify action agreement and local injectivity, and they find the replay traces when consumption is absent or split across domains.

For agent authorization, the practical rule is concise. Use the receipt as evidence of exact human approval. Let the authorization service decide whether that evidence is sufficient. Let the executor compare the exact action, consume the grant atomically, and then commit. Evidence can travel. Admission needs state.

Artifact Availability

The Tamarin models, exact tool versions, model hashes, and rerun commands are in the public EMILIA Protocol repository under `papers/eprint-action-bound-authorization`. The focused paper is a preprint candidate and has not been submitted to the IACR Cryptology ePrint Archive as of the date on its title page.

References

- [1] D. Dolev and A. C. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2), 1983.
- [2] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The Tamarin prover for the symbolic analysis of security protocols. In *CAV 2013*, LNCS 8044, 2013.
- [3] D. Fett, R. Küsters, and G. Schmitz. A comprehensive formal security analysis of OAuth 2.0. In *ACM CCS 2016*, 2016.
- [4] M. Barbosa, A. Boldyreva, S. Chen, and B. Warinschi. Provable security analysis of FIDO2. In *CRYPTO 2021*, LNCS 12827, pages 125–156, 2021.
- [5] P. Landrock and T. Pedersen. WYSIWYS? What you see is what you sign? *Information Security Technical Report*, 3(2):55–61, 1998.
- [6] A. Rundgren, B. Jordan, and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.

- [7] W3C. Web Authentication: An API for accessing public key credentials, Level 2. W3C Recommendation, April 2021.
- [8] T. Lodderstedt, J. Richer, and B. Campbell. OAuth 2.0 Rich Authorization Requests. RFC 9396, May 2023.
- [9] OpenID Foundation. OpenID Connect Client-Initiated Backchannel Authentication Flow, Core 1.0. September 2021.
- [10] J. Richer, P. Hunt, and M. B. Jones. Transaction Tokens. Internet-Draft, work in progress.
- [11] P. Kasselmann, J. Lombardo, Y. Rosomakho, B. Campbell, N. Steele, and A. Parecki. AI Agent Authentication and Authorization. Internet-Draft **draft-klrc-aiagent-auth-03**, work in progress, July 2026.
- [12] X. Weng. What You Approve Is What Executes: Consent Integrity for Black-Box LLM Agents. arXiv:2606.02668, June 2026.
- [13] D. Chaum. Blind signatures for untraceable payments. In *CRYPTO 1982*, 1983.
- [14] S. Brands. *Rethinking Public Key Infrastructures and Digital Certificates: Building in Privacy*. MIT Press, 2000.
- [15] I. Schrock. Authorization Receipts for High-Risk Agent Actions. Internet-Draft **draft-schrock-ep-authorization-receipts**, work in progress, August 2026.